CS-233 Project - Milestone 1: Predicting Gaming Addiction

Tommaso Sommaruga (407010) Steven Ji (392693) Tom Ravot (392844)

1 Introduction

This project aims to predict both the continuous addiction score and the categorical addiction level of users based on their gaming habits. The dataset consists of 2,000 samples with 13 features, divided into 1,600 samples for training and 400 for testing. We implemented Linear Regression for the regression task, Logistic Regression for classification, and K-Nearest Neighbors (KNN) which was adapted to perform both regression and classification tasks.

2 Method

2.1 Data Preparation

Exploratory analysis revealed significant class imbalance in the classification task. As visualized in Figure 1, Class 0 (low addiction) heavily dominates all splits of the dataset, while Class 2 (high addiction) remains a distinct minority.

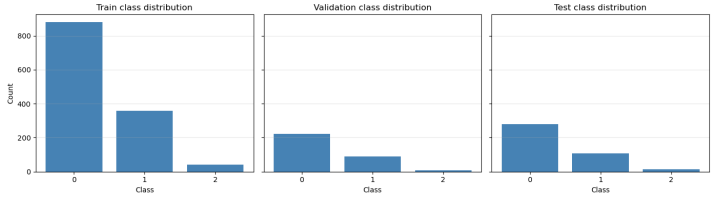


Figure 1: Class distribution across dataset splits, illustrating the severe imbalance

To address this without modifying the underlying dataset, we opted for internal model weighting rather than external sampling. Additionally:

- **Normalization:** Features were standardized using the training set mean (μ) and standard deviation (σ) for both train and test set: $x_{\text{norm}} = \frac{x-\mu}{\sigma}$
- **Bias Term:** A constant bias term of 1 was appended to the feature vector X.
- **Validation:** We utilized a 5-fold cross-validation strategy to search for the optimal combination of learning rate and weighting hyperparameters.

2.2 Linear Regression

For the regression task, we implemented a linear regression model. The relationship between the features X and the targets y is determined by finding the optimal weight vector w via the **closed-form formula**:

$$w = (X^T X)^{-1} X^T y \tag{1}$$

This analytical approach minimizes the Mean Squared Error (MSE) directly without requiring iterative optimization. Prior to fitting, features were standardized according to the procedure in Section 2.1, and a constant bias term of 1 was appended to the feature matrix X.

While we explored adding a Ridge regularization term λI to the $(X^T X)$ matrix to prevent potential collinearity, the feature matrix proved to be well-conditioned after normalization. Consequently, the standard solution ($\lambda = 0$) was maintained to avoid introducing unnecessary bias, yielding a competitive Test MSE of 0.993 (see in Figure 5).

2.3 Logistic Regression

For the multi-class classification task, we implemented multinomial logistic regression. The model computes class probabilities using the **Softmax** activation function:

$$P(y = c \mid \mathbf{x}) = \frac{\exp(\mathbf{x} \cdot \mathbf{w}_c)}{\sum_{j=1}^{C} \exp(\mathbf{x} \cdot \mathbf{w}_j)}$$

The model is trained by minimizing the Cross-Entropy loss via gradient descent. To address the class imbalance shown in Figure 1, we applied a `weighted_clipped` strategy. The gradient update incorporates sample weights α_i based on the inverse frequency of the classes:

$$\nabla \mathbf{W} = \frac{1}{\sum \alpha_i} \mathbf{X}^T ((\hat{\mathbf{Y}} - \mathbf{Y}) \odot \boldsymbol{\alpha})$$

The "clipping" mechanism ensures that no single minority sample exerts excessive influence on the gradient, preventing instability during training while still forcing the model to learn the features of Class 2.

2.4 K-Nearest Neighbors

- **Distance function:** Euclidean Distance.
- **Classification method:** majority vote, when there is a tie, choose the class whose samples have the smallest average distance to the test sample. Moreover, to deal with class imbalance, we introduce a class re-weighting.
- **Regression method:** if there exists a point such that distance equals to 0, return the value of that point; otherwise, return the average value of the k nearest neighbors, weighted by the inverse of the distance.

3 Experiments & Results

3.1 Experimental Setup

Performance is measured using Accuracy and Macro-F1 for classification, and Mean Squared Error (MSE) for regression. We utilize 5-fold cross-validation for hyperparameter tuning, measuring training and prediction latency.

3.2 Hyperparameter Search

3.2.1 KNN Classification

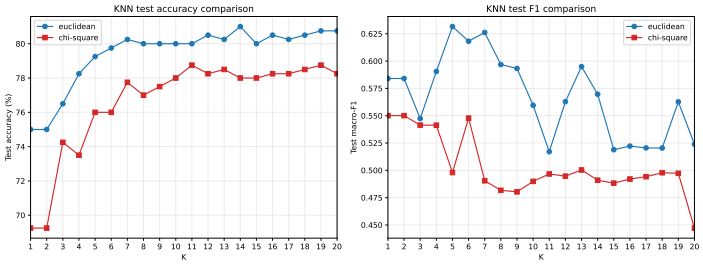


Figure 2: Test accuracy & Macro-F1 for KNN classification.

The Euclidean distance performs better than the Chi-Square in both metrics, and hence we keep the former. One could naively stop here, and choose the best K based on these results.

However, the accuracy is actually inflated due to class imbalance: by predominantly predicting class 0, the model achieves high accuracy despite poor performance on minority classes. Observe the low recall of class 2 (≈ 0.15, fig. 4), which means that most highly addicted people are not correctly identified. That is why we need class re-weighting.

With respect to this severe class imbalance, we use a strong correction formula: $w_c = \left(\frac{N}{n_c}\right)^{\alpha}, \quad \alpha \in [0, 1]$

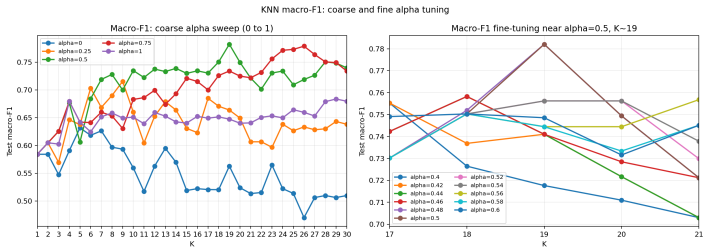


Figure 3: Selecting alpha.

The best performing alpha is 0.5 with K = 19 (additionally checked for K up to 50, and used cross-validation for close competitors). As a result, we achieved a highly favorable trade-off between the recall of C0 and the remaining metrics.

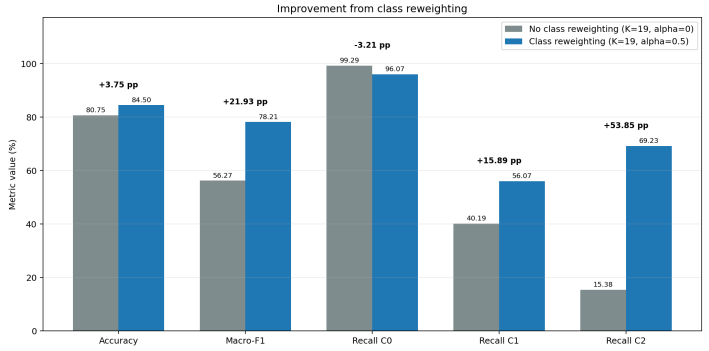


Figure 4: Final results & improvements.

3.2.2 Logistic Regression

We performed a grid search over the learning rate (η), iterations, and weighting strategies.

η	Iters	Weighting	CV Acc (%)	CV F1
0.00005	300	None	76.20	0.5123
0.00010	500	Weighted	78.40	0.6351
0.00050	**800**	**W-Clipped**	**79.75**	**0.6729**

Table 1: Logistic Regression grid search (5-fold CV).

The optimal configuration yielded a CV accuracy of $79.750\% \pm 1.970$ and a Macro-F1 of 0.672939 ± 0.0617.

3.3 Final Results on Test Set

Method	Acc (%)	Macro-F1	MSE
Linear Reg.	–	–	0.993
Logistic Reg.	81.750	0.714142	–
KNN (classif.)	84.500	0.782100	–
KNN (regress.)	–	–	1.800

Table 2: Test performance comparison.

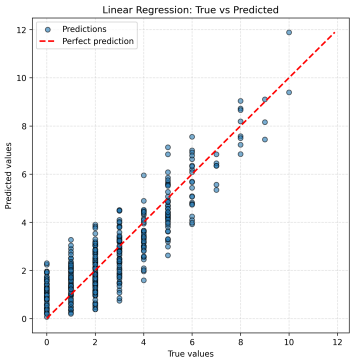


Figure 5: Actual vs. Predicted addiction scores

The graph above shows that the linear model's predictions closely follow the actual scores, confirming the low $MSE = 0.993$ obtained on the test set.

3.4 Runtime Analysis

While iterative training for Logistic Regression took about 0.042308, its prediction phase is near-instantaneous (0.000139s). In contrast, KNN has no theoretical training cost but suffers from significantly higher prediction latency (4.31s) as it must query the entire dataset for each test sample.

The Linear Regression model, utilizing a closed-form analytical solution, proved to be the most efficient method. It achieved a training time of 0.000267s and a prediction latency of 0.000007s, offering the fastest performance for real-time scoring.

Method	Training Time (s)	Prediction Time (s)
Linear Reg.	0.000267	0.000007
Logistic Reg.	0.042308	0.000139
KNN (classif.)	17.785878†	4.307074
KNN (regress.)	17.230738†	4.314654

Table 3: Runtime performance comparison.
†Training time for KNN reflects data indexing overhead in our implementation.

4 Discussion & Conclusion

Both models successfully addressed class imbalance via internal weighting (weighted_clipped and α-scaling) rather than external sampling. This preserved data integrity by avoiding synthetic artifacts.

Interestingly, for KNN, class re-weighting was the primary driver of performance; distance-based re-weighting had negligible impact and occasionally degraded results. While KNN achieved a higher F1 (0.78), Logistic Regression (0.71) offered a better efficiency-to-performance ratio. Overall, model-based weighting effectively identifies addiction classes without compromising data authenticity.

Furthermore, the regression task highlighted the strength of the Linear Regression model, which achieved a superior Test MSE of 0.993 compared to the 1.800 reached by KNN. This performance demonstrates that the closed-form analytical solution is highly effective for this dataset when features are properly normalized.

In conclusion, while KNN and Logistic Regression are valuable for classification, Linear Regression proved to be the most computationally efficient model, offering the lowest prediction latency. Overall, model-based weighting and analytical solvers effectively identify addiction patterns without compromising data authenticity.